FIAP

PROMPT ENGINEERING AND ARTIFICIAL INTELLIGENCE

Como Funcionam os LLMs Tokens, Embeddings e Arquitetura Transformer

Aula 03/18 — Módulo 1: Fundamentos de Inteligência Artificial

👨‍🏫 Prof. Jorge Luiz Gomes

⏱️ Duração: 2h

🐍 Google Colab · Sem API paga

Agenda da Aula

1	O que são Tokens? Tokenização: BPE e WordPiece	20 min
2	Embeddings — representações vetoriais e espaço semântico	15 min
3	Arquitetura Transformer: Self-Attention e "previsão do próximo token"	20 min
4	Parâmetros: temperatura, top-p e context window	10 min
5	Demo: tiktoken, sentence-transformers e GPT-2 com temperaturas	30 min
6	Casos Reais + Atividade da Aula	15 min

📚 **REFERÊNCIAS DESTA AULA**

Goodfellow et al., Cap. 10 · Vaswani et al. "Attention Is All You Need" (2017) · Brown et al. "Language Models are Few-Shot Learners" (2020)

01

Tokens e Tokenização

LLMs não leem palavras — eles leem tokens. Entender tokenização é entender o custo, a velocidade e o comportamento do modelo.

O que são Tokens?

Um **token** é a unidade básica que um LLM processa. Não é uma palavra, não é uma letra — é um **pedaço de texto** definido pelo tokenizador do modelo.

Exemplo de tokenização:

Frase: **"Inteligência Artificial é fascinante"**

`Int` `elig` `ência` `Art` `ificial` `é` `fasc` `in` `ante`

4 palavras → 9 tokens (em português, geralmente ~1.3 tokens por palavra)

Tipos de tokenização:

MÉTODO	COMO FUNCIONA	USADO POR
BPE (Byte-Pair Encoding)	Começa com caracteres individuais e vai fundindo os pares mais frequentes. Ex: "th"+"e" → "the"	GPT-2, GPT-3, GPT-4, LLaMA
WordPiece	Similar ao BPE, mas usa probabilidade ao invés de frequência para decidir as fusões	BERT, DistilBERT
SentencePiece	Trata o texto como sequência de bytes — funciona com qualquer idioma, até sem espaços	T5, Gemini, LLaMA 2+

💰 **POR QUE TOKENS IMPORTAM?**

Custo: APIs cobram por token (GPT-4: ~$30/1M tokens input). **Velocidade:** mais tokens = mais tempo de processamento. **Context window:** cada modelo tem um limite de tokens que pode processar de uma vez (GPT-4: 128K, Claude: 200K). **Prompt Engineering:** um prompt eficiente usa menos tokens para obter o mesmo resultado.

02

Embeddings

Como transformar palavras em números que capturam significado — o coração matemático da compreensão de línguagem.

Embeddings: Palavras como Vetores no Espaço

Um **embedding** (representação vetorial) é um vetor de números que representa o **significado** de uma palavra, frase ou documento. Palavras com significados similares ficam **próximas no espaço vetorial**.

A famosa analogia vetorial

`Rei - Homem + Mulher ≈ Rainha`

Operações matemáticas sobre embeddings capturam relações semânticas reais. Isso funciona porque o modelo aprendeu que a **diferença vetorial** entre "Rei" e "Homem" é similar à diferença entre "Rainha" e "Mulher".

Similaridade Cosseno

Para medir o quão similares dois textos são, calculamos o **cosseno do ângulo** entre seus vetores:

cos(θ) = 1.0 → idênticos
cos(θ) = 0.0 → sem relação
cos(θ) = -1.0 → opostos

Isso é a base de **busca semântica, RAG e sistemas de recomendação** (Aula 14).

🔗 **CONEXÃO COM PROMPT ENGINEERING**

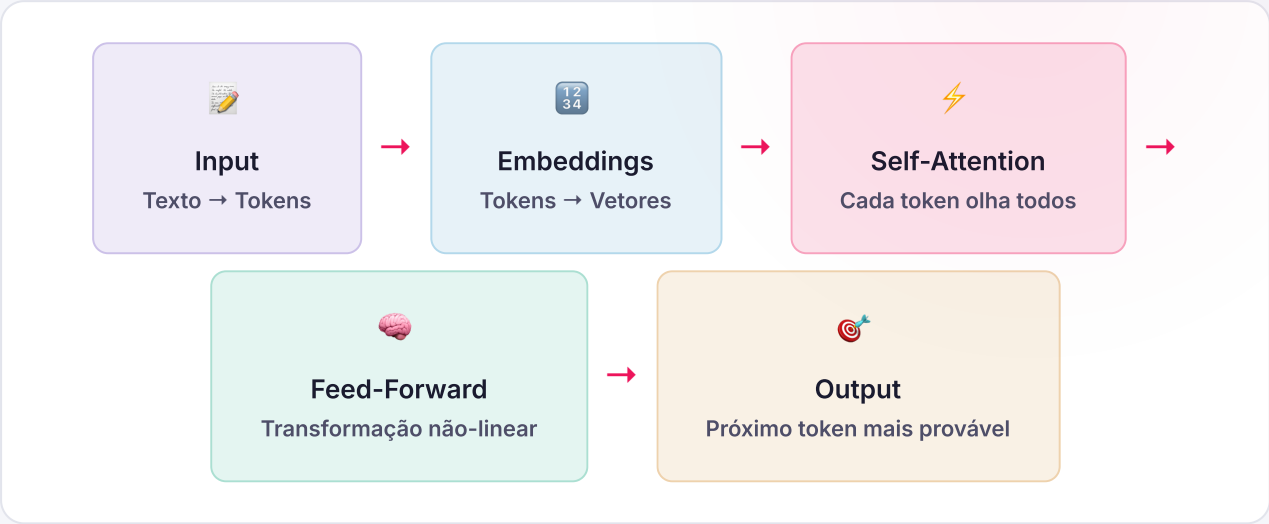
Quando você escreve um prompt, ele é convertido em embeddings antes de ser processado pelo modelo. Prompts semanticamente similares tendem a gerar respostas similares. Entender embeddings ajuda a entender por que **pequenas mudanças no prompt podem causar grandes mudanças no output**.

03

O Transformer

"Attention Is All You Need" — o paper de 2017 que mudou tudo. A arquitetura por trás de GPT, Claude, Gemini e LLaMA.

Arquitetura Transformer: A Base de Todos os LLMs



Self-Attention: A Inovação Fundamental

O **mecanismo de attention** permite que cada token "olhe" para **todos os outros tokens** da sequência simultaneamente, decidindo quais são relevantes para ele:

> 🔍 **EXEMPLO: "O BANCO ESTÁ PERTO DO RIO"**
>
> A palavra "banco" é ambígua — pode ser instituição financeira ou assento. O mecanismo de attention analisa as **outras palavras da frase** ("rio") e atribui um peso alto para "rio", concluindo que "banco" se refere a um assento, não ao Itaú. Sem attention, o modelo trataria cada palavra isoladamente.

Encoder-Decoder vs Decoder-Only

ARQUITETURA	COMO FUNCIONA	USADO POR	MELHOR PARA
Encoder-Decoder	Encoder compreende o input, decoder gera o output	T5, BART, modelos de tradução	Tradução, sumarização

Encoder-Only	Só compreende — classifica, extrai, compara	BERT, RoBERTa	Classificação, NER, busca
Decoder-Only	Só gera — prevê o próximo token autoregressivamente	**GPT, Claude, LLaMA, Gemini**	Geração de texto, chat, código

Como o LLM Gera Texto: Previsão do Próximo Token

Todo LLM generativo (GPT, Claude, LLaMA) faz **exatamente uma coisa**: dado uma sequência de tokens, prever qual é o **próximo token mais provável**. Repete isso em loop até gerar a resposta completa.

```
Input: O gato sentou no ___
Probabilidades:
"tapete" → 34%
"sofá" → 22%
"chão" → 18%
"telhado" → 8%
"carro" → 3%
... (milhares de opções com probabilidades menores)
```

Temperatura e Sampling

❄️ Temperatura 0.0

Sempre escolhe o token mais provável. **Determinístico** —

🌡️ Temperatura 0.7

Equilíbrio entre criatividade e coerência. **Padrão na**

🔥 Temperatura 1.5+

Tokens menos prováveis ganham mais

mesma pergunta, mesma resposta. Ideal para tarefas factuais.

maioria dos modelos. Bom para conversas e redação.

chance. **Criativo mas imprevisível**. Pode gerar nonsense.

04

Demo Prática

Tokenização com tiktoken, embeddings com sentence-transformers e geração com diferentes temperaturas.

Demo 1: Contando Tokens com tiktoken

01_tokenizacao.py · Google Colab

```
# 📦 Instalar tiktoken (tokenizador oficial da OpenAI)
!pip install tiktoken -q

import tiktoken

# 🔤 Usar o tokenizador do GPT-4 (cl100k_base)
enc = tiktoken.get_encoding("cl100k_base")

# 💬 Testar com frases em português e inglês
frases = [
    "Hello, world!",
    "Inteligência Artificial é fascinante",
    "O gato sentou no tapete vermelho",
    "def fibonacci(n): return n if n ≤ 1 else fibonacci(n-1) + fibonacci(n-
2)",
]

for frase in frases:
    tokens = enc.encode(frase)
    decoded = [enc.decode([t]) for t in tokens]
    print(f"\n📝 '{frase}'")
    print(f"   Tokens: {len(tokens)} → {decoded}")
```

💡 **OBSERVE**

Português gasta mais tokens que inglês para o mesmo conteúdo — os tokenizadores foram treinados majoritariamente com texto em inglês. Isso significa que prompts em

PT-BR são ligeiramente "mais caros" e ocupam mais da context window.

Demo 2: Embeddings e Similaridade Semântica

02_embeddings.py · Google Colab

```
# 📦 Instalar sentence-transformers
!pip install sentence-transformers -q

from sentence_transformers import SentenceTransformer, util

# 🧠 Carregar modelo multilíngue de embeddings
modelo = SentenceTransformer("all-MiniLM-L6-v2")

# 💬 Frases para comparar
frases = [
    "I love programming in Python",
    "Python is my favorite coding language",
    "The cat is sleeping on the couch",
    "I enjoy writing code",
]

# 🔢 Gerar embeddings (vetores de 384 dimensões)
embeddings = modelo.encode(frases)
print(f"Forma do vetor: {embeddings[0].shape}")  # (384,)

# 📐 Calcular similaridade entre todas as frases
similaridades = util.cos_sim(embeddings, embeddings)

for i in range(len(frases)):
    for j in range(i+1, len(frases)):
```

```
        print(f"'{frases[i][:35]}...' ↔ '{frases[j][:35]}...' =
{similaridades[i][j]:.3f}")
```

🤔 **O QUE ESPERAR?**

"I love programming in Python" e "Python is my favorite coding language" terão similaridade **alta (~0.85)** mesmo com palavras diferentes — porque o significado é o mesmo. Já "The cat is sleeping" terá similaridade **baixa (~0.15)** com as frases de programação.

Demo 3: GPT-2 com Diferentes Temperaturas

03_temperatura.py · Google Colab

```python
from transformers import pipeline

# ✍️ Carregar GPT-2 (124M parâmetros, gratuito)
gerador = pipeline("text-generation", model="gpt2")

prompt = "The future of artificial intelligence is"

# 🌡️ Testar com 3 temperaturas diferentes
temperaturas = [0.1, 0.7, 1.5]

for temp in temperaturas:
    resultado = gerador(
        prompt,
        max_new_tokens=60,
        temperature=temp,
        do_sample=True,
        top_p=0.95
    )
    print(f"\n🌡️ Temperatura {temp}:")
    print(f"   {resultado[0]['generated_text'][:250]}")
```

Execute 3 vezes e observe: com temperatura **0.1** o resultado é quase idêntico a cada vez; com **1.5**, cada execução gera algo completamente diferente (e às vezes incoerente).

Casos Reais: Benchmarks e Comparação de LLMs

🏆 BENCHMARKS PÚBLICOS

Como comparar modelos de forma justa

MMLU — 57 matérias acadêmicas, de história a cálculo. GPT-4 ~86%, Claude 3 ~85%, LLaMA 3 70B ~82%.
HumanEval — geração de código Python. GPT-4 ~91%, Claude 3 ~88%.
GPQA — perguntas de pós-graduação em ciência. Humanos especialistas: ~65%. GPT-4: ~39%.

Atenção: benchmarks não contam a história toda — o melhor modelo depende da tarefa.

🌍 CONTEXTO IMPORTA

Context Window: o diferencial de 2025-2026

GPT-4o: 128K tokens (~96K palavras → ~300 páginas de livro)
Claude: 200K tokens (~150K palavras → ~500 páginas)
Gemini 1.5: 1M tokens (livro inteiro)

Isso significa que você pode passar documentos enormes como contexto no prompt — a base para RAG e Context Engineering (Aulas 10 e 14).

🔑 TAKEAWAY PARA PROMPT ENGINEERING

Entender como LLMs funcionam por dentro — tokens, embeddings, attention, temperatura — é o que separa quem "usa IA" de quem **domina IA**. Você agora sabe por que o mesmo prompt dá resultados diferentes em modelos diferentes e com parâmetros diferentes.

Atividade da Aula

✏️

EXERCÍCIO PRÁTICO — INDIVIDUAL ⭐⭐☆

"Explorando Tokens, Embeddings e Temperatura"

Parte 1 — Tokenização (8 min)

1. Use o código de tiktoken para contar quantos tokens tem **seu nome completo**, uma **frase em português** e a **mesma frase traduzida para inglês**
2. Qual idioma gasta mais tokens? Por quê?

Parte 2 — Embeddings (10 min)

1. Usando o código de sentence-transformers, crie **5 frases**: 2 sobre um tema (ex: esportes) e 3 sobre outro tema (ex: culinária)
2. Calcule a similaridade. O modelo consegue distinguir os temas?

Parte 3 — Temperatura (7 min)

1. Usando o código de GPT-2, troque o prompt para algo da sua área de interesse
2. Execute com temperaturas 0.1, 0.7 e 1.5. Qual resultado é mais útil para sua tarefa?

💡 **Desafio bônus** ⭐⭐⭐ **:** Use matplotlib para plotar os embeddings em 2D com PCA. Importe `from sklearn.decomposition import PCA` e plote os vetores. As frases similares ficam próximas no gráfico?

Referências e Leitura Complementar

LIVRO **GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A.** Deep Learning. Pearson, 2017. Cap. 10 (Sequence Modeling).

PAPER **VASWANI, A. et al.** "Attention Is All You Need". NeurIPS, 2017. — O paper que criou o Transformer.

PAPER **BROWN, T. et al.** "Language Models are Few-Shot Learners" (GPT-3). NeurIPS, 2020.

ONLINE **OpenAI Tokenizer** — platform.openai.com/tokenizer — visualize tokens interativamente

ONLINE **Jay Alammar** — The Illustrated Transformer — melhor explicação visual do Transformer

ONLINE **sbert.net** — Sentence Transformers documentation

FIAP

Resumo da Aula 03

- ✅ Tokens são a unidade básica — PT-BR gasta mais tokens que inglês
- ✅ Embeddings capturam significado como vetores — base para busca semântica
- ✅ Transformer usa self-attention para entender contexto entre tokens
- ✅ LLMs preveem o próximo token — temperatura controla a criatividade
- ✅ Decoder-only (GPT, Claude, LLaMA) é a arquitetura dominante em 2026

→ PRÓXIMA AULA — AULA 04

Ambiente Python para IA — Colab, HF, Ollama e APIs Gratuitas

Lab completo: configurar tudo, rodar modelos locais e criar seu primeiro mini-chatbot.

Copyright © 2026 Prof. Jorge Luiz Gomes · FIAP · Todos os direitos reservados.